

TP PYTHON 2026

50 Projets Pratiques en Ligne de Commande

12 Débutant

13
Intermédiaire

13 POO

12 Expert

Python 3.13+ · CLI uniquement · POO systématique · Aucun code fourni

Introduction & Règles du Jeu

Ce TP couvre l'essentiel de Python 2026 à travers 50 projets exécutables en ligne de commande. Aucun code n'est fourni — seulement des énoncés, des contraintes, et des pistes de réflexion. La curiosité est ta meilleure alliée.

Règles fondamentales

- Chaque projet se lance avec : `python nom_projet.py`
- Documentation officielle Python autorisée : docs.python.org
- Chaque projet doit fonctionner sans interface graphique (CLI uniquement)
- Les modules de la bibliothèque standard sont autorisés sauf mention contraire
- À partir du niveau 3, la POO est systématiquement attendue et évaluée
- Ne passe pas au projet suivant sans avoir terminé le précédent
- Relis ton code 2 jours après l'avoir écrit : que changerais-tu ?

NIVEAU 1 — Débutant

Syntaxe, variables, conditions, boucles, fonctions

Projets 1 à 12

1

Calculatrice Classique Mathématiques / Interface CLI

Description

Crée une calculatrice interactive en ligne de commande. L'utilisateur entre deux nombres, choisit une opération (+, -, *, /, //, %, **), et le résultat s'affiche.

Contraintes

- Gérer la division par zéro avec un message explicite
- Proposer à l'utilisateur de recommencer après chaque calcul
- Afficher un menu clair avec les opérations disponibles
- Utiliser au minimum 5 fonctions distinctes (une par opération + menu + boucle principale)

Pistes de réflexion

- Comment Python gère-t-il les types int vs float lors d'une division ?
- Que se passe-t-il avec // sur des nombres négatifs ?
- Comment valider que l'entrée utilisateur est bien un nombre ?

2

Générateur de Tables de Multiplication Mathématiques / Formatage de texte

Description

L'utilisateur entre un nombre, et le programme affiche sa table de multiplication de 1 à 10 de façon propre et alignée.

Contraintes

- L'affichage doit être parfaitement aligné en colonnes (utiliser le formatage de chaînes Python)
- Proposer un mode "table complète" qui affiche toutes les tables de 1 à 10 côte à côte
- Permettre de choisir la limite (de 1 à N, N entré par l'utilisateur)
- Valider que l'entrée est un entier positif

Pistes de réflexion

- Explore la méthode `.format()` et les f-strings avec alignement (`<`, `>`)
- Quelle est la différence entre `str.zfill()` et le formatage avec `:` ?

3

Jeu du Nombre Mystère Jeu / Algorithmique

Description

Le programme choisit un nombre aléatoire entre 1 et 100. L'utilisateur doit le deviner. Après chaque tentative, le programme indique si le nombre cherché est plus grand ou plus petit.

Contraintes

- Afficher le nombre de tentatives à chaque essai
- Calculer et afficher un score basé sur le nombre de tentatives
- Sauvegarder le meilleur score dans un fichier texte `scores.txt` entre les sessions
- Proposer 3 niveaux de difficulté (1-50, 1-100, 1-1000)

Pistes de réflexion

- Quel module Python permet de générer un nombre aléatoire ?
- Comment lire et écrire dans un fichier texte en Python ?
- Quelle est la stratégie mathématiquement optimale pour deviner le nombre ?

4

Convertisseur d'Unités Sciences / Utilitaire

Description

Un convertisseur d'unités en ligne de commande qui prend en charge : températures (°C, °F, K), distances (km, miles, nm), poids (kg, lb, g).

Contraintes

- Menu principal avec 3 catégories, puis sous-menu par catégorie
- Toutes les conversions dans les deux sens
- Arrondir les résultats à 4 décimales
- Ne pas utiliser de bibliothèque externe (tout calculer manuellement)

Pistes de réflexion

- Comment structurer un programme avec plusieurs niveaux de menus ?
- Explore les dictionnaires pour stocker les formules de conversion

5

Analyse de Texte Traitement de texte / Statistiques

Description

L'utilisateur entre un texte (ou un nom de fichier .txt). Le programme affiche : nombre de mots, de phrases, de caractères, les 5 mots les plus fréquents, et le mot le plus long.

Contraintes

- Ignorer la casse (traiter "Python" et "python" comme le même mot)
- Ignorer la ponctuation lors du comptage des mots
- Afficher les résultats dans un tableau formaté en ASCII
- Accepter le texte via saisie directe ou via un fichier en argument (sys.argv)

Pistes de réflexion

- Comment nettoyer une chaîne de ses caractères spéciaux ?
- Explore le module collections.Counter
- Qu'est-ce que sys.argv et comment l'utiliser ?

6

Générateur de Mot de Passe Sécurité / Utilitaire

Description

Génère des mots de passe aléatoires selon les critères de l'utilisateur : longueur, majuscules, minuscules, chiffres, symboles.

Contraintes

- L'utilisateur choisit chaque critère individuellement (oui/non)
- Générer plusieurs mots de passe à la fois (l'utilisateur choisit combien)
- Évaluer et afficher la "force" du mot de passe (faible/moyen/fort/très fort)
- Permettre de copier le mot de passe dans le presse-papier (pyperclip autorisé)

Pistes de réflexion

- Quelle est la différence entre random et secrets ?
- Comment définir les critères d'un "bon" mot de passe ?
- Explore le module string (constantes de caractères)

7

Liste de Courses Intelligente Gestion de données / Fichiers

Description

Une application CLI pour gérer une liste de courses. L'utilisateur peut ajouter, supprimer, cocher des articles, voir la liste, et sauvegarder/charger depuis un fichier.

Contraintes

- Persistance des données dans un fichier JSON
- Regrouper les articles par catégorie (Fruits, Légumes, Épicerie...)
- Afficher la liste avec des cases cochées/non cochées en ASCII ([x] / [])
- Possibilité d'exporter la liste en .txt formaté

Pistes de réflexion

- Pourquoi JSON est-il préférable à un simple .txt pour ce cas ?
- Explore json.load() et json.dump()
- Comment gérer l'absence du fichier au premier lancement ?

8

Horloge et Chronomètre Temps / Interface CLI

Description

Un programme avec 3 modes : affichage de l'heure actuelle, chronomètre (start/stop/reset), et compte à rebours avec alerte.

Contraintes

- Mode chronomètre en temps réel (mise à jour dans le terminal)
- Le compte à rebours prend une durée en entrée (format mm:ss)
- Affichage "en direct" dans le terminal sans créer de nouvelles lignes (utiliser \r)
- Sauvegarder les temps du chronomètre dans un historique

Pistes de réflexion

- Explore les modules time et datetime
- Comment effacer une ligne dans le terminal sans effacer tout l'écran ?
- Quelle est la différence entre time.time() et time.perf_counter() ?

9

Quiz Culturel Éducation / Jeux

Description

Un quiz en ligne de commande avec des questions à choix multiples. Les questions sont chargées depuis un fichier JSON. L'utilisateur choisit un thème et une difficulté.

Contraintes

- Fichier de questions en JSON avec format standardisé
- Mélanger les questions à chaque partie (ordre aléatoire)
- Mélanger aussi l'ordre des réponses proposées
- Afficher un récapitulatif détaillé à la fin
- Score final avec lettre (A, B, C, D, F)

Pistes de réflexion

- Conçois toi-même le format JSON de tes questions
- Comment mélanger une liste avec random.shuffle() ?
- Explore random.sample() vs random.shuffle()

10

Calendrier ASCII Date / Affichage formaté

Description

Affiche un calendrier mensuel en ASCII dans le terminal pour n'importe quel mois/année. Permet de naviguer entre les mois.

Contraintes

- Afficher les jours correctement alignés (lundi en premier)
- Mettre en évidence le jour actuel avec des crochets [15]
- Permettre d'ajouter des événements liés à une date (sauvegardés en JSON)
- Afficher les jours avec événements avec un marqueur *

Pistes de réflexion

- Explore le module calendar (il existe déjà !)
- Pourquoi ne pas en recoder un de zéro pour comprendre comment il fonctionne ?
- Comment déterminer le premier jour d'un mois donné ?

11

Traducteur de César Cryptographie / Algorithmique**Description**

Implémente le chiffre de César : décale chaque lettre de l'alphabet d'une valeur N choisie par l'utilisateur. Encode et décode des messages.

Contraintes

- Préserver la casse (majuscules restent majuscules)
- Ignorer les chiffres et symboles (les laisser inchangés)
- Proposer un mode "force brute" qui teste tous les décalages possibles (0-25)
- Afficher la table de correspondance lettre → lettre pour le décalage choisi

Pistes de réflexion

- Qu'est-ce que `ord()` et `chr()` ? Comment fonctionnent-ils ?
- Ce chiffrement est-il "sécurisé" ? Pourquoi ?
- Comment automatiser la détection de la langue pour identifier le bon décalage ?

12

Calculateur IMC & Santé Santé / Calcul**Description**

Calcule l'IMC à partir du poids et de la taille. Affiche la catégorie OMS correspondante et des informations générales.

Contraintes

- Accepter les données en unités métriques ET impériales
- Afficher une barre de progression ASCII indiquant où se situe l'IMC
- Calculer et afficher le poids idéal selon la formule de Lorentz
- Sauvegarder un historique des mesures (date + IMC) dans un fichier CSV

Pistes de réflexion

- Explore le module `csv` pour écrire dans un fichier CSV
- Comment afficher une barre de progression avec des caractères ASCII ?
- Quelle est la différence entre IMC et composition corporelle ?

NIVEAU 2 — Intermédiaire

Structures de données, fichiers, modules, exceptions

Projets 13 à 25

13 Gestionnaire de Contacts Gestion de données / CRUD

Description

Une mini base de données de contacts en ligne de commande : ajouter, modifier, supprimer, rechercher des contacts. Chaque contact a : nom, prénom, téléphone, email, groupe.

Contraintes

- Persistance en JSON
- Recherche par nom partiel (insensible à la casse)
- Filtrage par groupe
- Exportation en CSV
- Importation depuis CSV (avec gestion des doublons)
- Tri par différents critères (nom, groupe, date d'ajout)

Pistes de réflexion

- Comment structurer un projet avec plusieurs fichiers .py ?
- Explore argparse pour passer des arguments en ligne de commande
- Pense à la gestion des erreurs : que se passe-t-il si le fichier JSON est corrompu ?

14 Éditeur de Notes Markdown Productivité / Fichiers

Description

Un gestionnaire de notes en ligne de commande. Chaque note est un fichier Markdown. L'utilisateur peut créer, lister, rechercher, afficher et supprimer des notes.

Contraintes

- Stocker les notes dans un dossier ~/notes/
- Rendu simplifié : **texte** → majuscules, # titre → ligne avec === dessous
- Recherche full-text dans le contenu de toutes les notes
- Afficher les notes triées par date de modification
- Tags dans les métadonnées YAML de chaque fichier

Pistes de réflexion

- Explore os.path et pathlib pour manipuler les chemins
- Quelle est la différence entre os.listdir() et pathlib.Path.iterdir() ?
- Comment lire les métadonnées YAML sans bibliothèque externe ?

15 Simulateur de Dés RPG Jeux / Aléatoire

Description

Un simulateur de lancers de dés pour jeux de rôle. Supporte la notation standard : 2d6, 1d20, 4d6-drop-lowest, 1d8+3.

Contraintes

- Parser la notation dés avec des expressions régulières
- Mode "avantage/désavantage" (lance deux fois, garde le meilleur/pire)
- Historique des lancers dans la session
- Statistiques : résultats min/max/moyen sur N lancers simulés
- Afficher la distribution des résultats en histogramme ASCII

Pistes de réflexion

- Explore le module `re` pour les expressions régulières
- Comment dessiner un histogramme horizontal avec des caractères ASCII ?
- Quelle est la distribution attendue pour 2d6 ? Pourquoi ?

16 Système de Fichiers TODO [Productivité / CLI](#)

Description

Un gestionnaire de tâches TODO inspiré de taskwarrior. Les tâches ont : titre, priorité, date limite, projet, statut.

Contraintes

- Interface entièrement via `argparse` (pas de menu interactif)
- Ex: `python todo.py add "Finir le TP" --priority high --project python`
- Affichage couleur dans le terminal (via codes ANSI, sans bibliothèque)
- Filtrage par projet, priorité, statut
- Tâches expirées mises en évidence automatiquement

Pistes de réflexion

- Explore `argparse` avec sous-commandes (`add`, `list`, `done`, `delete`)
- Comment afficher du texte coloré dans un terminal avec des codes ANSI ?
- Que sont les codes ANSI `\033[31m` (rouge), `\033[32m` (vert), etc. ?

17 Détecteur de Palindrome & Anagramme [Algorithmique / Texte](#)

Description

Analyse des mots et phrases : détecte les palindromes, trouve les anagrammes dans un dictionnaire, génère toutes les anagrammes possibles d'un mot.

Contraintes

- Ignorer les espaces et accents pour les palindromes de phrases
- Utiliser un fichier dictionnaire (.txt)
- Afficher les anagrammes trouvés dans le dictionnaire
- Mesurer et afficher le temps d'exécution de chaque opération
- Mode "challenge" : timer pour trouver le maximum d'anagrammes

Pistes de réflexion

- Quelle structure de données est la plus efficace pour chercher des anagrammes ?
- Explore `unicodedata.normalize()` pour gérer les accents
- Comment mesurer le temps d'exécution d'une fonction en Python ?

18 Mini Interpréteur de Commandes [☞ Système / Architecture](#)

Description

Crée un shell minimal avec ses propres commandes intégrées : ls, cd, pwd, mkdir, touch, cat, echo, help, exit.

Contraintes

- Boucle REPL (Read-Eval-Print Loop) principale
- Gestion des erreurs (fichier non trouvé, permissions, etc.)
- Historique des commandes navigable (module readline)
- Support des chemins relatifs et absolus
- Commande history qui affiche les dernières commandes

Pistes de réflexion

- Explore les modules os et shutil
- Qu'est-ce qu'un REPL ? Donne d'autres exemples
- Explore le module readline pour l'autocomplétion

19 Analyseur de Logs [☞ DevOps / Traitement de fichiers](#)

Description

Analyse un fichier de logs (format Apache/Nginx ou syslog). Affiche des statistiques : IP les plus actives, URLs les plus visitées, codes d'erreur.

Contraintes

- Parser les logs avec des expressions régulières
- Détecter les patterns suspects (trop de requêtes depuis une même IP)
- Générer un rapport résumé en texte
- Filtrage par plage horaire, code HTTP, IP
- Exporter les résultats en CSV

Pistes de réflexion

- Trouve un exemple de fichier de log Apache sur internet
- Comment parser une ligne de log avec re.match() ?
- Qu'est-ce qu'une IP suspecte dans un contexte web ?

20 Simulateur de Banque Simple [☞ Finance / Logique métier](#)

Description

Simule un compte bancaire avec : dépôt, retrait, virement entre comptes, historique des transactions, calcul d'intérêts.

Contraintes

- Multi-comptes (chaque compte a un IBAN fictif généré)
- Historique complet des transactions en JSON
- Calcul d'intérêts mensuels (taux paramétrable)
- Solde interdit négatif (sauf compte "découvert autorisé")
- Relevé de compte exportable en .txt formaté

Pistes de réflexion

- Explore le module decimal pour les calculs financiers (pourquoi pas float ?)
- Comment générer un IBAN fictif valide en termes de format ?
- Qu'est-ce que l'intérêt composé ? Comment le calculer ?

21 Jeu de Pendu [📁 Jeu / Algorithmique](#)

Description

Le classique jeu du pendu en CLI avec dessin ASCII du pendu, liste de mots dans plusieurs catégories, et indices disponibles.

Contraintes

- Dessin du pendu en ASCII qui se construit progressivement
- Mots chargés depuis un fichier JSON catégorisé
- Un indice disponible (définition ou catégorie) qui coûte une vie
- Statistiques : taux de victoire, mots les plus ratés
- Mode "multijoueur local" : un joueur entre le mot, l'autre devine

Pistes de réflexion

- Comment effacer et redessiner l'état du jeu à chaque tour ?
- Quelle est la meilleure stratégie pour le pendu ? (fréquence des lettres)
- Cherche la fréquence des lettres en français/anglais

22 Simulateur de Tri Visualisé [📁 Algorithmique / Visualisation](#)

Description

Implémente et visualise en ASCII plusieurs algorithmes de tri : Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort.

Contraintes

- Visualisation ASCII : barres verticales de hauteur proportionnelle (▮)
- Animation pas-à-pas ou continue selon la vitesse choisie
- Compter et afficher le nombre de comparaisons et d'échanges
- Permettre de comparer deux algorithmes côte à côte
- Mode "benchmark" : mesure le temps réel pour N éléments

Pistes de réflexion

- Qu'est-ce que la complexité algorithmique ($O(n)$, $O(n^2)$, $O(n \log n)$) ?
- Pourquoi Merge Sort est-il plus efficace que Bubble Sort ?
- Comment rafraîchir l'écran proprement dans le terminal ?

23 Générateur de Recettes [📁 Alimentation / Génération de données](#)

Description

Un programme qui gère une base de recettes : ajouter, chercher par ingrédients, calculer les valeurs nutritionnelles, adapter les quantités.

Contraintes

- Base de recettes en JSON

- Recherche "qu'est-ce que je peux cuisiner avec : œufs, farine, sucre ?"
- Adaptation automatique des quantités au nombre de personnes
- Calcul nutritionnel basique via une table de valeurs JSON
- Génération d'une liste de courses à partir d'un menu de la semaine

Pistes de réflexion

- Comment modéliser une recette en JSON ?
- Explore les sets Python pour trouver l'intersection entre ingrédients
- Qu'est-ce qu'une fraction en Python ? (module fractions)

24 Système de Votes et Sondages [Statistiques / Démocratie](#)

Description

Crée et gère des sondages en CLI. Plusieurs méthodes de vote : majoritaire simple, Borda, approbation. Affiche les résultats avec visualisation.

Contraintes

- Créer/sauvegarder/charger des sondages en JSON
- Implémenter les 3 méthodes de vote (cherche leur définition !)
- Afficher les résultats en diagramme bâtons ASCII
- Détecter et afficher les paradoxes (ex: paradoxe de Condorcet)
- Générer un rapport comparatif des résultats selon les méthodes

Pistes de réflexion

- Qu'est-ce que le paradoxe de Condorcet ?
- Pourquoi différentes méthodes de vote peuvent donner des résultats différents ?
- Comment représenter un vote par classement vs un vote par approbation ?

25 Simulateur de Réseau Social (CLI) [Réseaux / Graphes](#)

Description

Simule un mini réseau social : utilisateurs, connexions (amis), publications, fil d'actualité. Tout en CLI, persistance JSON.

Contraintes

- Créer des profils, suivre des utilisateurs, publier des messages
- Fil d'actualité : messages des personnes suivies, triés par date
- Algorithme de suggestion d'amis (amis d'amis)
- Statistiques : utilisateur le plus suivi, chemin le plus court entre deux utilisateurs
- Export du graphe de connexions au format texte

Pistes de réflexion

- Comment représenter un graphe en Python (liste d'adjacence vs matrice) ?
- Qu'est-ce que le BFS (Breadth-First Search) ? Utilise-le pour le chemin le plus court
- Explore collections.deque pour le BFS

NIVEAU 3 — POO & Architecture

Classes, héritage, polymorphisme, design patterns

Projets 26 à 38

⚠ À partir d'ici, chaque projet DOIT utiliser la Programmation Orientée Objet de façon structurée. Des classes, des méthodes, de l'héritage. Pas de code spaghetti.

26

Bibliothèque de Livres (POO Fondations) POO / Gestion

Description

Gère une bibliothèque de livres avec un système d'emprunts. Premier projet POO : modélise tout avec des classes.

Contraintes

- Classes obligatoires : Livre, Membre, Bibliotheque, Emprunt
- Livre : titre, auteur, ISBN, disponible, genre
- Membre : nom, ID, liste des emprunts actifs, historique
- Bibliotheque : collection de livres, liste de membres, méthodes CRUD
- Emprunt : livre, membre, date_début, date_retour_prévue
- Persistance JSON, recherche multi-critères
- `__str__` et `__repr__` définis sur chaque classe

Pistes de réflexion

- Quelle est la différence entre `__str__` et `__repr__` ?
- Qu'est-ce que l'encapsulation ? Utilise des attributs privés (`_attribut`)
- Comment sérialiser un objet en JSON ? (méthode `to_dict()`)

27

Zoo Virtuel (Héritage & Polymorphisme) POO / Héritage

Description

Simule un zoo. Chaque animal a des comportements communs, mais des implémentations différentes. C'est la démonstration parfaite de l'héritage et du polymorphisme.

Contraintes

- Classe abstraite Animal avec méthodes abstraites : `se_nourrir()`, `faire_du_bruit()`, `se_deplacer()`
- Au minimum 6 classes filles : Lion, Aigle, Dauphin, Serpent, Pingouin, Tortue
- Classe Zoo qui gère une collection d'animaux
- Chaque journée simulée : les animaux mangent, font du bruit, se déplacent
- Classe Soigneur qui peut interagir avec les animaux

Pistes de réflexion

- Explore le module `abc` (Abstract Base Classes)
- Qu'est-ce que le polymorphisme ? Donne un exemple concret
- Quelle est la différence entre "is-a" et "has-a" en POO ?

28

RPG en Ligne de Commande (POO Avancée) Jeu / POO

Description

Un mini RPG au tour par tour. Personnages, monstres, inventaire, combats, progression de niveau.

Contraintes

- Classe abstraite Entite (base pour Personnage et Monstre)
- Système d'inventaire avec classe Item et sous-classes (Arme, Armure, Potion)
- Personnage avec : points de vie, mana, niveau, expérience, inventaire
- Système de combat au tour par tour avec classe Combat
- Sauvegarde/chargement de la partie (sérialisation complète des objets)
- Génération procédurale de donjons simples (rooms et connexions)

Pistes de réflexion

- Explore le pattern "Composition over Inheritance"
- Comment sauvegarder un objet complexe ? Explore pickle vs JSON
- Qu'est-ce que la génération procédurale ? Comment générer un donjon simple ?

29

Système Bancaire Orienté Objet Finance / Design Patterns

Description

Refactorise le projet 20 en POO complète, en ajoutant des types de comptes différents et un système de notifications.

Contraintes

- Classe abstraite Compte avec sous-classes : CompteCourant, CompteEpargne, ComptePro
- Chaque type de compte a des règles différentes (taux, limites, frais)
- Pattern Observer : système de notifications (alerte si solde < seuil)
- Classe Banque qui centralise la gestion
- Classe Transaction immuable (utilise @property et pas de setter)
- Historique des transactions avec pattern Iterator

Pistes de réflexion

- Qu'est-ce que le Design Pattern Observer ? Implémente-le en Python
- Qu'est-ce qu'un objet immuable ? Comment le créer en Python ?
- Explore les @property, @setter, @deleter en Python

30

Moteur de Règles (Pattern Strategy) Architecture / Design Patterns

Description

Crée un moteur de règles configurable. Applications : validation de formulaire, calcul de prix avec remises, filtrage de données.

Contraintes

- Pattern Strategy : interchangeable entre différents algorithmes
- Les règles sont configurables (chargées depuis JSON ou définies en code)
- Chaîne de règles : les règles s'exécutent en séquence

- Implémenter pour deux cas d'usage : validateur + calculateur de prix
- Tests unitaires pour chaque règle (module unittest)

Pistes de réflexion

- Qu'est-ce que le Design Pattern Strategy ?
- Qu'est-ce qu'un test unitaire ? Explore le module unittest
- Comment séparer la logique métier de la logique de présentation ?

31 Simulateur de Flotte de Véhicules Transport / POO

Description

Gère une flotte de véhicules : voitures, camions, motos, vélos électriques. Simule des trajets, la consommation, la maintenance.

Contraintes

- Hiérarchie : Vehicule → VehiculeMoteur → VoitureEssence, VoitureElectrique, Camion
- Interfaces (classes abstraites) : Rechargeable, Refuelable, Electrique
- Simulation de trajet avec consommation calculée selon le type et la distance
- Système de maintenance : kilométrage, alertes entretien
- Rapport de flotte : coûts, efficacité, émissions CO2

Pistes de réflexion

- Explore l'héritage multiple en Python
- Qu'est-ce qu'un Mixin ? Comment l'utiliser ?
- Comment modéliser l'héritage quand un objet a plusieurs "natures" ?

32 Système de Plugins (Architecture Extensible) Architecture / Métaprogrammation

Description

Crée une application extensible par plugins. L'application de base est un "transformateur de texte", et les plugins ajoutent des transformations.

Contraintes

- Mécanisme de découverte automatique de plugins (fichiers dans un dossier plugins/)
- Interface de plugin standardisée (classe abstraite Plugin)
- Plugins fournis : uppercase, lowercase, inverser, compter mots, encoder base64, chiffrer César
- L'utilisateur peut chaîner les plugins : input | uppercase | inverser | base64
- Système de configuration par plugin

Pistes de réflexion

- Explore importlib pour charger dynamiquement des modules Python
- Qu'est-ce que l'introspection en Python ? (dir(), getattr(), hasattr())
- Explore __init_subclass__ pour l'enregistrement automatique de plugins

33 Éditeur de Graphes (Théorie des Graphes) Mathématiques / Graphes / POO

Description

Un programme pour créer, manipuler et analyser des graphes (orientés et non-orientés). Visualisation ASCII.

Contraintes

- Classes : Graphe, Sommet, Arete
- Algorithmes implémentés : DFS, BFS, Dijkstra, détection de cycles
- Visualisation ASCII du graphe (pour les petits graphes)
- Vérification : connexité, bipartisme, arbre couvrant minimal
- Export au format DOT (format Graphviz)

Pistes de réflexion

- Implémente Dijkstra sans regarder la solution : réfléchis à la logique
- Quelle est la différence entre un graphe orienté et non-orienté ?
- Explore heapq pour la file de priorité dans Dijkstra

34

Simulateur de Système d'Exploitation (Processus) [Systèmes / Architecture](#)

Description

Simule un ordonnanceur de processus. Les processus ont une priorité, un temps d'exécution, des états (prêt, en cours, bloqué, terminé).

Contraintes

- Classe Processus avec : PID, nom, priorité, burst_time, état
- Classe Ordonnanceur avec plusieurs algorithmes : FCFS, Round Robin, Priorité
- Simulation pas-à-pas avec affichage de l'état à chaque tick
- Calcul des métriques : temps d'attente moyen, temps de réponse, efficacité
- Diagramme de Gantt en ASCII

Pistes de réflexion

- Qu'est-ce que l'ordonnancement de processus dans un OS ?
- Qu'est-ce que le Round Robin et pourquoi est-il équitable ?
- Explore heapq pour la file de priorité

35

Compilateur de Mini-Langage [Langages / Théorie](#)

Description

Implémente un interpréteur pour un mini-langage de script supportant : variables, opérations arithmétiques, conditions if/else, boucles while.

Contraintes

- Phases : Lexer (tokenisation) → Parser (AST) → Évaluateur
- Classes : Token, Lexer, AST_Node, Parser, Interpreter
- Gestion des erreurs avec messages clairs (ligne, caractère, type d'erreur)
- Le mini-langage peut calculer fibonacci et factorielle
- Mode debug qui affiche les tokens et l'AST

Pistes de réflexion

- Qu'est-ce qu'un token ? Qu'est-ce qu'un AST (Abstract Syntax Tree) ?
- Explore les concepts de grammaire formelle

→ Comment Python lui-même est-il interprété ? (c'est exactement ce que tu fais !)

36

Application de Messagerie Locale (Multi-threading) [Concurrence / Réseau local](#)

Description

Une application de "chat" en ligne de commande, client-serveur sur réseau local (ou localhost), utilisant le multi-threading.

Contraintes

- Serveur : accepte plusieurs connexions simultanées (un thread par client)
- Client : envoie et reçoit des messages en temps réel
- Noms d'utilisateur uniques, salons de discussion
- Chiffrement simple des messages (XOR ou César)
- Logging de toutes les connexions et messages côté serveur

Pistes de réflexion

- Explore les modules socket et threading
- Qu'est-ce qu'un deadlock ? Comment l'éviter ?
- Explore `threading.Lock()` pour protéger les données partagées

37

Moteur de Recherche Local [Recherche d'information / Indexation](#)

Description

Indexe un répertoire de fichiers textes et permet une recherche full-text rapide avec classement par pertinence.

Contraintes

- Index inversé : mot → liste de fichiers qui le contiennent
- Score TF-IDF simplifié pour le classement (cherche la formule !)
- Recherche avec opérateurs : AND, OR, NOT, guillemets pour phrase exacte
- Mise à jour incrémentale de l'index
- Extrait de contexte : affiche la phrase contenant le terme trouvé

Pistes de réflexion

- Qu'est-ce que TF-IDF et pourquoi est-il efficace ?
- Qu'est-ce qu'un index inversé ? (C'est la base de Google !)
- Explore pickle pour persister l'index

38

Système de Versioning Simplifié (Mini Git) [DevOps / Architecture](#)

Description

Implémente un système de contrôle de version ultra-simplifié, inspiré de Git : init, add, commit, log, diff, checkout.

Contraintes

- init : crée le dossier .minigit/
- add : stage les fichiers (copie les snapshots)

- commit : enregistre un snapshot avec message et timestamp
- log : affiche l'historique des commits
- diff : compare deux versions d'un fichier
- Chaque commit a un hash unique (MD5 du contenu suffit)

Pistes de réflexion

- Explore hashlib pour calculer des hashes
- Comment Git stocke-t-il réellement les fichiers ? (cherche "Git internals")
- Explore difflib pour comparer des fichiers

NIVEAU 4 — Avancé & Expert

Async, métaprogrammation, performances, architecture complète

Projets 39 à 50

39 Web Scraper Asynchrone Web / Async

Description

Scrape plusieurs sites en parallèle avec asyncio et aiohttp. Extrait des données structurées et les stocke.

Contraintes

- Utiliser asyncio + aiohttp (pas de requests synchrone)
- Scrape au minimum 3 sources différentes en parallèle
- Rate limiting : ne pas surcharger les serveurs (max N requêtes/seconde)
- Retry automatique avec backoff exponentiel en cas d'échec
- Export des données en JSON et CSV

Pistes de réflexion

- Quelle est la différence entre parallélisme et concurrence ?
- Qu'est-ce que asyncio.gather() ?
- Pourquoi le scraping asynchrone est-il plus rapide que synchrone ?

40 Décorateurs et Métaprogrammation Python avancé / Métaprogrammation

Description

Crée une bibliothèque de décorateurs utiles, et explore la métaprogrammation avec les métaclases.

Contraintes

- Décorateurs : @timer, @retry(n), @cache, @validate_types, @singleton
- @validate_types vérifie les types des arguments au runtime
- @singleton garantit qu'une classe n'a qu'une seule instance
- Métaclasse AutoProperty qui crée automatiquement des propriétés depuis les annotations
- Tous les décorateurs doivent préserver la signature (functools.wraps)

Pistes de réflexion

- Qu'est-ce qu'un décorateur ? Comment fonctionne-t-il réellement ?
- Qu'est-ce qu'une métaclasse ? (__class__, type, __new__, __init__)
- Explore functools.wraps et pourquoi c'est important

41 Système de Cache Intelligent Performance / Architecture

Description

Implémente plusieurs stratégies de cache : LRU, LFU, TTL (Time To Live), avec statistiques et éviction.

Contraintes

- Classe abstraite Cache avec sous-classes : LRUCache, LFUCache, TTLCache
- Décorateur @cached(strategy="lru", maxsize=128, ttl=60) réutilisable
- Statistiques : hit rate, miss rate, évictions
- Persistance optionnelle du cache sur disque
- Benchmark comparatif des stratégies sur une fonction lente simulée

Pistes de réflexion

- Qu'est-ce que LRU (Least Recently Used) et LFU (Least Frequently Used) ?
- Explore `functools.lru_cache` : c'est quoi sous le capot ?
- Explore `collections.OrderedDict` pour implémenter LRU

42

Interpréteur de Requêtes SQL Simplifié Base de données / Parsing

Description

Implémente un mini-moteur SQL qui exécute des requêtes SELECT/INSERT/UPDATE/DELETE sur des "tables" stockées en mémoire.

Contraintes

- Parser les requêtes SQL basiques : SELECT ... FROM ... WHERE ... ORDER BY ... LIMIT ...
- Support de JOIN (INNER JOIN entre deux tables)
- Support des agrégats : COUNT, SUM, AVG, MAX, MIN, GROUP BY
- Persistance des tables en JSON/CSV
- Optimisations : index sur une colonne, explain plan (affiche le plan d'exécution)

Pistes de réflexion

- Comment un vrai moteur SQL parse-t-il une requête ?
- Qu'est-ce qu'un index et pourquoi accélère-t-il les requêtes ?
- Explore l'AST qu'on doit construire pour une requête SQL

43

Framework Web Minimaliste Web / Architecture

Description

Crée un micro-framework web en Python (comme Flask, mais en miniature). Routing, requêtes, réponses, templates basiques.

Contraintes

- Serveur HTTP basique avec `http.server`
- Décorateur `@app.route('/path', methods=['GET', 'POST'])`
- Parsing des paramètres GET et du body POST (JSON et form-data)
- Moteur de templates basique (variables `{{var}}`, boucles `{% for %}`)
- Middleware : logger, CORS headers, rate limiting

Pistes de réflexion

- Explore `http.server.BaseHTTPRequestHandler`
- Comment Flask implémente-t-il le routing ? (Explore son code source !)
- Qu'est-ce qu'un middleware dans le contexte d'un serveur web ?

44

Analyseur de Code Python Outils de développement / AST

Description

Un outil d'analyse statique de code Python : complexité cyclomatique, dépendances, couverture de code, style.

Contraintes

- Utiliser le module ast pour parser le code Python
- Calculer la complexité cyclomatique de chaque fonction
- Détecter les imports inutilisés, les variables non utilisées
- Générer un rapport de qualité de code
- Visualiser l'arbre des dépendances entre fichiers

Pistes de réflexion

- Explore le module ast : ast.parse(), ast.walk(), ast.NodeVisitor
- Qu'est-ce que la complexité cyclomatique ?
- Python peut analyser son propre code : c'est fascinant, non ?

45

Système de Workflow (Pipeline Pattern) Architecture / Data Engineering

Description

Un moteur de pipeline de traitement de données. Les données passent à travers une série d'étapes configurables.

Contraintes

- Classes : Pipeline, Step, Context (données qui passent entre les étapes)
- Steps prédéfinis : Filter, Transform, Aggregate, Split, Merge, Cache
- Configuration du pipeline en JSON ou via API fluide : pipeline.add(step1).add(step2)
- Exécution parallèle des steps indépendants
- Monitoring : temps par step, données traitées, erreurs

Pistes de réflexion

- Qu'est-ce que le Pattern Pipeline ? Donne des exemples concrets
- Comment détecter les dépendances entre steps pour paralléliser ?
- Explore concurrent.futures.ThreadPoolExecutor

46

Simulateur de Machine Neuronale Simple Intelligence Artificielle / Mathématiques

Description

Implémente un réseau de neurones de A à Z (sans bibliothèque ML), entraîne-le sur un problème simple (XOR ou classification).

Contraintes

- Classe NeuralNetwork avec nombre de couches/neurones configurable
- Forward pass, backward pass (backpropagation) implémentés manuellement
- Fonctions d'activation : sigmoid, ReLU, tanh
- Visualisation de l'entraînement (loss en temps réel en ASCII)
- Sauvegarde/chargement des poids du réseau

Pistes de réflexion

- Qu'est-ce que la rétropropagation (backpropagation) ?
- Pourquoi le XOR ne peut pas être résolu par un réseau à une seule couche ?
- Explore numpy pour les opérations matricielles (autorisé ici !)

47

Interpréteur de Bytecode Python Python interne / Bas niveau

Description

Explore et visualise le bytecode Python. Crée un désassembleur et un interpréteur de bytecode simple.

Contraintes

- Utiliser le module `dis` pour désassembler du code Python
- Créer un visualiseur de pile d'exécution (stack) pas-à-pas
- Implémenter l'évaluation d'un sous-ensemble d'opcodes
- Comparer le bytecode de différentes façons d'écrire la même chose
- Explorer les optimisations automatiques du compilateur Python

Pistes de réflexion

- Qu'est-ce que le bytecode ? Quelle est la différence avec le code machine ?
- Explore `compile()`, `exec()`, `eval()` en Python
- Qu'est-ce que CPython ? Explore son code source (écrit en C !)

48

Base de Données Clé-Valeur Persistante Base de données / Systèmes

Description

Implémente une base de données clé-valeur persistante (comme Redis simplifié) avec un protocole texte simple.

Contraintes

- Opérations : SET key value, GET key, DEL key, EXISTS key, KEYS pattern, TTL key
- Persistance : WAL (Write-Ahead Log) + snapshots périodiques
- Serveur TCP avec protocole texte (telnet-compatible)
- Support des types : string, list, hash, set
- Transactions : MULTI / EXEC / DISCARD

Pistes de réflexion

- Qu'est-ce qu'un WAL (Write-Ahead Log) et pourquoi est-il important ?
- Comment Redis persiste-t-il les données (RDB vs AOF) ?
- Qu'est-ce qu'une transaction dans une base de données ?

49

Système de Détection d'Anomalies Data Science / Statistiques

Description

Détecte des anomalies dans des séries temporelles (données de monitoring, prix, logs) avec plusieurs algorithmes statistiques.

Contraintes

- Algorithmes : Z-Score, IQR, Moving Average + seuil, Isolation Forest simplifié
- Génération de données simulées avec anomalies injectées
- Visualisation ASCII des séries temporelles avec anomalies marquées
- Comparaison des algorithmes : precision, recall, F1
- Dashboard ASCII en temps réel simulant un flux de données

Pistes de réflexion

- Qu'est-ce que le Z-Score ? Quand est-il inapproprié ?
- Explore statistics (module standard) pour les calculs statistiques
- Qu'est-ce que precision et recall ? Pourquoi est-ce important ?

50

Projet Intégrateur : Plateforme de Dev Personnel Architecture complète / Projet final

Description

Crée une plateforme personnelle de développement en CLI : gestionnaire de projets, time tracker, notes techniques, snippets de code, statistiques de productivité.

Contraintes

- Architecture en couches : CLI → Services → Repositories → Stockage
- Design Patterns obligatoires : Repository, Service Layer, Observer, Command
- Base de données SQLite (module sqlite3) — pas de JSON cette fois
- Authentification basique (PIN ou mot de passe hashé avec hashlib)
- Système de plugins (réutiliser ce que tu as fait au projet 32)
- Projets : créer, archiver, tags, statut
- Time Tracker : start/stop sur un projet, rapport hebdomadaire
- Notes : markdown, tags, liées à un projet
- Snippets : code multi-langage, recherche par tag/langage

Pistes de réflexion

- Comment organiser une codebase qui grandit ? (structure de dossiers)
- Explore sqlite3 : pourquoi SQL est préférable à JSON à ce stade ?
- Qu'est-ce que la "clean architecture" ? Lis l'article fondateur
- Comment tester une application avec des dépendances externes ? (mocking)
- Réfléchis : qu'aurais-tu fait différemment sur tes premiers projets ?

Annexe — Ressources Essentielles 2026

Documentation officielle

- Python 3.13+ : docs.python.org/3/
- PEP 8 (style guide) : peps.python.org/pep-0008/
- What's New in Python 3.13 : à lire impérativement

Modules standard à maîtriser

os, sys, pathlib, json, csv, re, datetime, collections, itertools, functools, abc, typing, dataclasses, asyncio, threading, socket, sqlite3, hashlib, unittest, argparse, logging

Concepts Python 2026 à explorer

- Type Hints & annotations (PEP 484, PEP 563, PEP 696)
- Dataclasses (@dataclass, field(), __post_init__)
- Pattern Matching (match/case, introduit en Python 3.10)
- Walrus Operator (:=, PEP 572)
- f-strings avancées (PEP 701, débogage avec =)
- TypeVar, Generic, Protocol pour le typage avancé
- asyncio moderne (TaskGroups, ExceptionGroups)

Conseils de progression

- Ne passe pas au projet suivant sans avoir terminé le précédent
- Relis ton code 2 jours après l'avoir écrit : que changerais-tu ?
- Utilise `python -m py_compile fichier.py` pour vérifier la syntaxe
- Utilise `pylint` ou `ruff` pour améliorer ton style
- Écris au moins 3 tests unitaires pour chaque projet à partir du niveau 3